

JavaScript

Funções, Arrays e Objetos

Da lógica solta da Aula 4 a um código organizado — hoje, tudo dentro de um único arquivo HTML, construído com muito código guiado, ao vivo, em sala.

DISCIPLINA

Programação — Web Coding (CC/P2)

MINISTRADO POR

Prof. Esp. Daniel Costa

index.html

```
<script>
  function calcularMedia(notas) {
    return (notas[0] + notas[1]) / 2;
  }
  console.log(calcularMedia([7, 8]));
</script>
```

O que vamos construir em Web Coding

Ainda no bloco 04 — JavaScript. Hoje aprofundamos a linguagem: funções, arrays e objetos.

01

Web

Como a Internet e as aplicações Web funcionam.

02

HTML5

A estrutura. O esqueleto de qualquer página.

03

CSS

A aparência. Cores, layout, responsividade.

04

JavaScript

O comportamento. Funções e estruturas de dados — hoje.

05

Bootstrap

Agilidade. Componentes prontos e responsivos.

06

JSON + SQLite

Os dados. Armazenar, consultar, persistir.

Hoje: ainda no navegador, mas reunindo **HTML e JavaScript num único arquivo** — e com muito código guiado, passo a passo, em sala.

De onde partimos hoje

Na Aula 4, o Sistema de Notas funcionou — mas toda a lógica ficou solta, dentro de um único `for`.

O QUE JÁ TEMOS

Lógica básica funcionando

- Variáveis, tipos, operadores, expressões.
- `if/else` para decidir; `for/while` para repetir.
- Tudo escrito uma única vez, sem reaproveitar.

O QUE MUDA HOJE

Organização e estrutura de dados

- Funções: blocos de código reutilizáveis, com nome.
- Arrays: listas de valores, com métodos prontos.
- Objetos: dados agrupados por propriedades nomeadas.

Mesmo problema do Sistema de Notas — só que agora resolvido de um jeito que qualquer programador reconheceria como "**código limpo**".

Por que usar funções?

Uma função é um bloco de código com nome, que você pode chamar quantas vezes quiser.

SEM FUNÇÃO

Código repetido

- Calcular a média de 3 alunos = escrever a mesma conta 3 vezes.
- Um erro no cálculo precisa ser corrigido em vários lugares.

COM FUNÇÃO

Código reutilizável

- Escreve o cálculo uma vez, chama quantas vezes precisar.
- Corrigir um erro conserta todos os usos de uma vez.

Essa ideia tem nome: **DRY** — *Don't Repeat Yourself* ("não se repita"). É um dos princípios mais importantes de programação estruturada.

Declarando e chamando uma função

Declarar é definir o que a função faz. Chamar é mandar executar.

```
funcao.js

function saudar() {
  console.log("Olá, turma!");
}
saudar();
saudar();

> Olá, turma!
> Olá, turma!
```

`function nome() { ... }` só declara a função — o código dentro só roda quando você a chama com `nome()`.

Parâmetros: dados de entrada

Parâmetros deixam a função genérica — o mesmo código funciona para valores diferentes.

parametros.js

```
function saudar(nome) {  
  console.log(`Olá, ${nome}!`);  
}  
saudar("Ana");  
saudar("Bruno");
```

PARÂMETRO

nome

É uma variável que só existe dentro da função.

ARGUMENTO

"Ana", "Bruno"

O valor real passado na chamada.

VÁRIOS

a, b, c...

Uma função pode ter mais de um parâmetro, separados por vírgula.

Parâmetros com valor padrão

Quando quem chama a função não informa um argumento, um valor padrão evita que o resultado vire undefined.

parametro-padrao.js

```
function saudar(nome = "visitante") {  
  console.log(`Olá, ${nome}!`);  
}  
saudar("Ana");  
saudar();
```

```
> Olá, Ana!  
> Olá, visitante!
```

nome = "visitante" só é usado quando o argumento não é passado. É uma forma simples de deixar a função mais segura — vamos usar isso no projeto de hoje.

return: devolvendo um resultado

return faz a função devolver um valor para quem a chamou — sem imprimir nada sozinha.

```
retorno.js

function calcularMedia(n1, n2) {
  return (n1 + n2) / 2;
}
const media = calcularMedia(7, 9);
console.log(media); // 8
```

RETURN

Encerra e devolve

Encerra a função e devolve o valor indicado.

GUARDAR

numa variável

O retorno pode ser guardado numa variável, como media.

DIFERENÇA

log ≠ return

console.log só mostra; return devolve o valor para ser usado depois.

Arrow functions: a forma moderna

Uma sintaxe mais curta para escrever funções — muito comum no JavaScript atual.

```
arrow.js

// forma tradicional
function dobrar(n) {
  return n * 2;
}

// arrow function equivalente
const dobrar = (n) => n * 2;

// com dois parâmetros
const somar = (a, b) => a + b;
```

Sem chaves e sem return explícito, uma arrow function de uma linha já devolve o resultado automaticamente. Vamos usar muito esse formato com arrays daqui a pouco.

Criando e acessando um array

Um array guarda uma lista de valores, em ordem, acessados por um número (índice) começando em 0.

```
arrays.js

const notas = [7, 8.5, 6];
console.log(notas[0]); // 7 - primeiro item
console.log(notas[2]); // 6 - terceiro item
console.log(notas.length); // 3 - quantidade de itens
```

O primeiro item está no índice **0**, não no 1. É o erro mais comum de quem está começando com arrays.

Adicionando e removendo itens

Métodos prontos alteram o array sem precisar recriá-lo do zero.

metodos.js

```
const notas = [7, 8];
notas.push(9);    // adiciona no fim: [7, 8, 9]
notas.pop();      // remove o último: [7, 8]
notas.unshift(5); // adiciona no início: [5, 7, 8]
```

MÉTODO	FAZ
<code>push()</code>	Adiciona um item no final do array.
<code>pop()</code>	Remove e devolve o último item.
<code>unshift()</code>	Adiciona um item no início do array.
<code>shift()</code>	Remove e devolve o primeiro item.

Percorrendo com forEach

forEach executa uma função para cada item do array — substitui o for manual na maioria dos casos.

```
foreach.js

const notas = [7, 8, 5];
notas.forEach((nota) => {
  console.log(`Nota: ${nota}`);
});
```

```
> Nota: 7
> Nota: 8
> Nota: 5
```

forEach recebe uma arrow function como parâmetro — ela é chamada uma vez para cada item, automaticamente.

map e filter: transformando dados

Dois métodos que criam um novo array a partir do original, sem alterá-lo.

map-filter.js

```
const notas = [5, 7, 9, 4];
const comBonus = notas.map((n) => n + 1);
console.log(comBonus); // [6, 8, 10, 5]
const aprovados = notas.filter((n) => n >= 6);
console.log(aprovados); // [7, 9]
```

MÉTODO	DEVOLVE
<code>map()</code>	Um array do mesmo tamanho, com cada item transformado.
<code>filter()</code>	Um array menor ou igual, só com os itens que passam no teste.

Objetos: dados com nome

Enquanto o array organiza por posição (índice), o objeto organiza por nome (propriedade).

```
objeto.js

const aluno = {
  nome: "Ana",
  idade: 20,
  notas: [7, 8],
};
console.log(aluno.nome);      // Ana
console.log(aluno["idade"]);  // 20
```

`objeto.propriedade` (notação de ponto) é a forma mais comum de acessar um valor. `objeto["propriedade"]` funciona igual e é útil quando o nome vem de uma variável.

Alterando e adicionando propriedades

Um objeto não é fixo — você pode mudar um valor existente ou criar uma propriedade nova a qualquer momento.

objeto-alterar.js

```
const aluno = { nome: "Ana", idade: 20 };  
aluno.idade = 21;           // altera um valor existente  
aluno.email = "ana@exemplo.com"; // cria uma propriedade nova  
console.log(aluno);
```

```
> { nome: 'Ana', idade: 21, email: 'ana@exemplo.com' }
```

Isso vai ser útil no projeto de hoje: dá para guardar o resultado calculado **de volta no próprio objeto** do aluno.

Arrays de objetos

A combinação mais usada em aplicações reais: uma lista onde cada item é um objeto completo.

lista-objetos.js

```
const alunos = [
  { nome: "Ana", notas: [7, 8] },
  { nome: "Bruno", notas: [5, 4] },
];
alunos.forEach((aluno) => {
  console.log(aluno.nome, aluno.notas);
});
```

Isso é exatamente uma "tabela" na cabeça de um programador: cada objeto é uma linha, cada propriedade é uma coluna.

Uma função, uma responsabilidade

Programação estruturada significa dividir um problema grande em pequenas funções, cada uma fazendo uma coisa só.

TUDO JUNTO

Difícil de ler e testar

- Um bloco gigante calcula, decide e imprime ao mesmo tempo.
- Um erro em qualquer parte é difícil de localizar.

DIVIDIDO EM FUNÇÕES

Fácil de ler e testar

- `calcularMedia()` só calcula.
- `verificarSituacao()` só decide.
- Cada função pode ser testada e corrigida sozinha.

É exatamente essa divisão que vamos aplicar agora, ao vivo, evoluindo o Sistema de Notas da Aula 4.

Sistema de Notas 2.0

A mesma ideia da Aula 4, agora organizada em funções e usando os métodos de array de hoje.

FUNCTION

calcularMedia()

Recebe as notas de um aluno e devolve a média.

FUNCTION

verificarSituacao()

Recebe a média e devolve "aprovado" ou "reprovado".

ARRAY

Lista de alunos

Array de objetos, cada um com nome e notas.

FOREACH

Processar a turma

Percorre a lista chamando as duas funções para cada aluno.

Diferença de hoje: nada de `index.html` + `script.js` separados. O JavaScript vai morar **dentro do próprio HTML**, numa tag `<script>`. Vamos construir isso juntos, passo a passo.

Passo 1 — Um arquivo, HTML e script juntos

Um único index.html guarda a página inteira — a tag `<script>` entra antes do `</body>`.

```
index.html

<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <title>Sistema de Notas</title>
</head>
<body>
  <h1>Sistema de Notas 2.0</h1>
  <p>Abra o console do navegador (F12) para ver os resultados.</p>

  <script>
    // todo o JavaScript da aula vai aqui dentro
  </script>
</body>
</html>
```

Tudo entre `<script>` e `</script>` é executado pelo navegador como JavaScript — exatamente como antes no `script.js`, só que agora no mesmo arquivo.

Passo 2 — Função calcularMedia

Primeira peça: uma função que recebe as notas de um aluno e devolve a média.

index.html — dentro do <script>

```
function calcularMedia(notas) {  
  return (notas[0] + notas[1]) / 2;  
}
```

Escreva essa função dentro do <script> criado no Passo 1 — nada muda em como uma função funciona, só o lugar onde ela mora.

Passo 3 — Função verificarSituacao

Segunda peça: uma função que recebe a média e devolve a situação do aluno.

index.html — dentro do <script>

```
function calcularMedia(notas) {  
  return (notas[0] + notas[1]) / 2;  
}  
  
function verificarSituacao(media) {  
  return media ≥ 6 ? "aprovado" : "reprovado";  
}
```

Passo 4 — Array de alunos (objetos)

Terceira peça: a turma inteira, como um array de objetos — a estrutura de dados de hoje.

index.html — dentro do <script>

```
function calcularMedia(notas) { /* Passo 2 */ }  
function verificarSituacao(media) { /* Passo 3 */ }  
  
const alunos = [  
  { nome: "Ana", notas: [7, 8] },  
  { nome: "Bruno", notas: [5, 4] },  
  { nome: "Carla", notas: [9, 10] },  
];
```

As funções dos Passos 2 e 3 continuam ali, só encolhidas no slide para caber a novidade. Ninguém apaga nada — só acrescenta.

Passo 5 — `forEach` juntando tudo

Última peça: percorrer a turma, chamando as duas funções para cada aluno.

```
index.html — dentro do <script>

/* ...funções e array dos passos anteriores... */

alunos.forEach((aluno) => {
  const media = calcularMedia(aluno.notas);
  const situacao = verificarSituacao(media);
  console.log(`${aluno.nome}: ${situacao} (${media})`);
});
```

Arquivo completo — index.html

HTML e JavaScript, lado a lado, em um único arquivo.

index.html — completo

```
<body>
  <h1>Sistema de Notas 2.0</h1>
  <p>Abra o console (F12) para ver os resultados.</p>

  <script>
    function calcularMedia(notas) {
      return (notas[0] + notas[1]) / 2;
    }
    function verificarSituacao(media) {
      return media ≥ 6 ? "aprovado" : "reprovado";
    }
    const alunos = [
      { nome: "Ana", notas: [7, 8] },
      { nome: "Bruno", notas: [5, 4] },
      { nome: "Carla", notas: [9, 10] },
    ];
    alunos.forEach((aluno) => {
      const media = calcularMedia(aluno.notas);
      const situacao = verificarSituacao(media);
      console.log(`${aluno.nome}: ${situacao} (${media})`);
    });
  </script>
</body>
```

Checklist de entrega

Antes de considerar pronto, confira se o seu arquivo cobre todos os itens abaixo.

- ✓ Arquivo único `index.html` com HTML e `<script>` juntos
- ✓ Função `calcularMedia(notas)` usando um array como parâmetro
- ✓ Função `verificarSituacao(media)` com `return`
- ✓ Lista de alunos como array de objetos
- ✓ Uso de `forEach` para percorrer a turma (sem `for` manual)
- ✓ `console.log` mostrando nome, situação e média de cada aluno
- ✓ Nenhum erro no console ao abrir o arquivo

Guarde esse formato — funções pequenas e bem definidas, tudo organizado em um único arquivo. É exatamente essa base que você vai usar nas próximas aulas.

Sintaxe vista hoje

Uma colinha rápida com o vocabulário JavaScript desta aula.

SINTAXE	CATEGORIA	FUNÇÃO
<code>function nome() {}</code>	Funções	Declara um bloco de código reutilizável
<code>parâmetro = valor</code>	Funções	Define um valor padrão quando o argumento não é passado
<code>return</code>	Funções	Devolve um valor para quem chamou a função
<code>(x) ⇒ x * 2</code>	Funções	Arrow function — forma curta de escrever funções
<code>array[indice]</code>	Arrays	Acessa um item pela posição, começando em 0
<code>push / pop / unshift / shift</code>	Arrays	Adicionam ou removem itens do início/fim
<code>forEach</code>	Arrays	Executa uma função para cada item
<code>map / filter</code>	Arrays	Criam um novo array transformado ou filtrado
<code>objeto.propriedade</code>	Objetos	Acessa ou altera um valor pelo nome
<code>array de objetos</code>	Objetos	Lista onde cada item tem várias propriedades

Quatro ideias que resolvem o resto

01

Função é a unidade de reuso do código

Escreva uma vez, chame quantas vezes precisar.

02

return devolve; console.log só mostra

São coisas diferentes — confundir os dois é um erro comum.

03

forEach substitui o for na maioria dos casos

Mais curto e mais legível para percorrer arrays.

04

HTML e JavaScript podem viver no mesmo arquivo

A tag `<script>` roda dentro da própria página.

FIM DA AULA 5

Obrigado.

Hoje o código ganhou organização de verdade — funções, arrays e objetos trabalhando juntos num único arquivo. Na próxima aula, 28/09, é dia de Bootstrap. Depois, em 05/10, a Avaliação 1.

DÚVIDAS SÃO BEM-VINDAS

Prof. Esp. Daniel Costa